

## Lekcja 14 – to szybko nadrobić

### Temat: Instrukcja DQL (ang. Data Querying Language) - SELECT, order by, LIMIT, OFFSET, WHERE, GROUP BY

```
DROP TABLE IF EXISTS pracownicy;
```

```
-- Tworzenie tabeli
```

```
CREATE TABLE pracownicy (  
    imie          VARCHAR(50) NOT NULL,  
    nazwisko      VARCHAR(50) NOT NULL,  
    departament  VARCHAR(50),  
    pensja        DECIMAL(10,2)  
);
```

```
-- Wstawianie danych
```

```
INSERT INTO pracownicy (imie, nazwisko, departament, pensja) VALUES  
( 'Jan',      'Kowalski',      'IT',      5000.00),  
( 'Anna',     'Nowak',          'HR',      4500.00),  
( 'Piotr',    'Zieliński',    'IT',      6200.00),  
( 'Maria',    'Wiśniewska',   'Finanse', 7000.00),  
( 'Tomasz',   'Lewandowski',  'IT',      5500.00),  
( 'Katarzyna', 'Kamińska',    'HR',      5200.00),  
( 'Adam',     'Nowicki',      'Finanse', 4800.00),  
( 'Ola',      'Dąbrowska',    'IT',      6500.00),  
( 'Robert',   'Malinowski',   'IT',      NULL);
```

## ORDER BY

**Instrukcja ORDER BY w zapytaniu SELECT** służy do **sortowania** wyników zapytania. Bez niej wiersze są zwracane w **nieokreślonej kolejności** (zależnej od silnika bazy danych i fizycznego układu danych na dysku). ORDER BY jest **ostatnią** klauzulą w zapytaniu (przed LIMIT/OFFSET).

Składnia:

```
SELECT kolumny  
FROM tabela
```

```
[WHERE warunek]
ORDER BY kolumna1 [ASC|DESC], kolumna2 [ASC|DESC], ...
```

- **ASC** – rosnąco (domyślne, można pominąć)
- **DESC** – malejąco

### Przykłady:

```
-- Pracownicy posortowani alfabetycznie po nazwisku (rosnąco)
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko;
```

```
-- To samo, ale malejąco
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko DESC;
```

```
SELECT imie, nazwisko, pensja, pensja * 1.1 AS nowa_pensja
FROM pracownicy
ORDER BY nowa_pensja DESC;          -- można użyć aliasu
```

```
-- lub po pozycji kolumny w SELECT (niezalecane, ale działa)
SELECT imie, nazwisko, pensja, pensja * 1.1 AS nowa_pensja
FROM pracownicy
ORDER BY 3 DESC;                    -- sortuje po 3. kolumnie (pensja)
```

### Sortowanie z NULL

Różne bazy danych traktują *NULL* inaczej:

**PostgreSQL, SQL Server, SQLite, MySQL, MariaDB** - *NULL* jako najmniejsza wartość (na początku przy ASC)

### Inne sposoby sortowania:

**ORDER BY RANDOM()** - losowa kolejność (PostgreSQL)

**ORDER BY 1, 2** - sortowanie po pierwszej i drugiej kolumnie

### LIMIT i OFFSET

LIMIT 3 OFFSET 4 → weź 3, pomiń 4

LIMIT 3, 4 → pomiń 3, weź 4

*- Sposób 1 (z OFFSET – bardziej czytelny)*

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko
LIMIT 3 OFFSET 4;
```

*-- Sposób 2 (z przecinkiem – najpopularniejszy)*

```
SELECT imie, nazwisko, pensja
FROM pracownicy
ORDER BY nazwisko
LIMIT 4, 3;      -- UWAGA: najpierw offset, potem count!
```

## WHERE

WHERE w MySQL to **filtr wierszy** – działa **przed** grupowaniem (GROUP BY) i mówi bazie: „pokaż tylko te rekordy, które spełniają warunek”.

Składnia:

```
SELECT kolumny
FROM tabela
WHERE warunek          -- ← tutaj filtrujemy
[GROUP BY ...]
[HAVING ...]
ORDER BY ...;
```

### Kolejność wykonywania zapytania:

1. FROM (skąd dane)
2. WHERE ← filtrujemy wiersze
3. GROUP BY
4. HAVING
5. SELECT, ORDER BY, LIMIT

### Najważniejsze operatory w WHERE

| Operator                                                                        | Znaczenie                     | Przykład                                             |
|---------------------------------------------------------------------------------|-------------------------------|------------------------------------------------------|
| <code>=</code>                                                                  | równa się                     | <code>departament = 'IT'</code>                      |
| <code>!=</code> lub <code>&lt;&gt;</code>                                       | różne od                      | <code>pensja != 5000</code>                          |
| <code>&gt;</code> / <code>&lt;</code> / <code>&gt;=</code> / <code>&lt;=</code> | porównanie liczb              | <code>pensja &gt; 6000</code>                        |
| <code>LIKE</code>                                                               | wyszukiwanie tekstu (z % i _) | <code>nazwisko LIKE 'K%'</code>                      |
| <code>IN</code>                                                                 | w liście wartości             | <code>departament IN ('IT', 'HR')</code>             |
| <code>BETWEEN</code>                                                            | w zakresie                    | <code>pensja BETWEEN 5000 AND 6000</code>            |
| <code>IS NULL</code> / <code>IS NOT NULL</code>                                 | NULL-e                        | <code>pensja IS NULL</code>                          |
| <code>AND</code> / <code>OR</code> / <code>NOT</code>                           | łączenie warunków             | <code>pensja &gt; 5000 AND departament = 'IT'</code> |

## Przykłady:

### 1. Prosty filtr – tylko jeden dział

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE departament = 'IT';
```

### 2. Więcej niż jedna warunek (AND)

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE departament = 'IT'
AND pensja > 6000;
```

### 3. Albo jeden, albo drugi (OR)

```
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE departament = 'HR'
OR departament = 'Finanse';
```

### 4. Wyszukiwanie tekstowe (LIKE)

```
SELECT imie, nazwisko
FROM pracownicy
WHERE nazwisko LIKE 'K%';
-- albo
WHERE nazwisko LIKE '%ski';
```

### 5. Zakres pensji

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE pensja BETWEEN 5000 AND 6000;
```

## 6. Rekordy z NULL

```
SELECT imie, nazwisko
FROM pracownicy
WHERE pensja IS NULL;
```

## 7. Kilka działów naraz (IN)

```
SELECT imie, nazwisko, departament
FROM pracownicy
WHERE departament IN ('IT', 'HR');
```

## Różnica WHERE vs HAVING

- WHERE – filtruje **pojedyncze wiersze** (przed grupowaniem)
- HAVING – filtruje **całe grupy** (po GROUP BY)

```
-- WHERE filtruje przed grupowaniem
SELECT departament, COUNT(*)
FROM pracownicy
WHERE pensja > 5000          -- tylko osoby z pensją > 5000
GROUP BY departament;
```

```
-- HAVING filtruje po grupowaniu
SELECT departament, AVG(pensja)
FROM pracownicy
GROUP BY departament
HAVING AVG(pensja) > 5000;   -- tylko działy ze średnią > 5000
```

## GROUP BY

GROUP BY w MySQL służy do **grupowania wierszy**, które mają te same wartości w wybranych kolumnach, a potem wykonywania na tych grupach operacji agregujących (np. liczenie, sumowanie, średnia).  
Bez GROUP BY agregaty (COUNT, SUM, AVG...) **liczą wszystko razem**.  
Z GROUP BY – liczą **osobno dla każdej grupy**.

### Przykłady.

#### 1. Ile osób pracuje w każdym dziale? (najpopularniejsze użycie)

```
SELECT
    departament,
    COUNT(*) AS liczba_pracownikow
```

```
FROM pracownicy
GROUP BY departament;
```

## 2. Średnia pensja w każdym dziale

```
SELECT
    departament,
    ROUND(AVG(pensja), 2) AS srednia_pensja,
    COUNT(*) AS ile_osob
FROM pracownicy
GROUP BY departament;
```

## 3. Suma wszystkich pensji w każdym dziale

```
SELECT
    departament,
    SUM(pensja) AS suma_pensji,
    MIN(pensja) AS najnizsza,
    MAX(pensja) AS najwyzsza
FROM pracownicy
GROUP BY departament;
```

## 4. Tylko działy, w których pracuje więcej niż 3 osoby (HAVING)

```
SELECT
    departament,
    COUNT(*) AS liczba
FROM pracownicy
GROUP BY departament
HAVING liczba > 3;          -- filtrujemy grupy!
```

## 5. Filtrujemy przed grupowaniem (WHERE)

```
SELECT
    departament,
    COUNT(*) AS ile,
    AVG(pensja) AS srednia
FROM pracownicy
WHERE pensja IS NOT NULL      -- pomijamy rekord z NULL
GROUP BY departament
HAVING AVG(pensja) > 5000;
```

## 6. Grupowanie po więcej niż jednej kolumnie

```
SELECT
    departament,
    COUNT(*) AS ile_osob
FROM pracownicy
GROUP BY departament, imie;
```